Definitions and Introduction

The final unit that we shall discuss in this course is NP Completeness. To begin our discussion of this topic we must first begin shifting our focus from "algorithms" to "problems". We could try and formally define these two ideas to give us a rigorous understand of what they mean, but that is not necessary for our purposes. We need only get a "feel" of the topic of NP Completeness in this course. Therefore, a problem (for our purposes) is essentially just a mathematically posed question. Some examples of problems that we have seen in those course are

Fibonacci Number Problem Given that $f_0 = 0$, $f_1 = 1$, find the value of $f_n = f_{n-1} + f_{n-2}$.

Searching Problem Given an array A, determine if the value K is in the array.

Sorting Problem Given an array A of size n, find a permutation A' of the array such that $A'[1] \leq A'[2] \leq \cdots \leq A'[n]$.

Minimum Spanning Tree Problem Given a connected, edge-weighted undirected graph find a minimum spanning tree of the graph.

Shortest Path problem Problem Given a connected, edge-weighted undirected graph and a pair of vertices v_1 and v_n in the graph, find the shortest path from v_1 to v_n.

With this definition of a problem we can see that an algorithm is a procedure which solves a problem. For example, we solved the Fibonacci Number Problem recursively, the Searching Problem (as written) was solved with iterative search, the Sorting Problem was solved in many ways, the Minimum Spanning Tree Problem was solved in two ways by Prim's and Kurskal's algorithm, and the Shortest Path Problem was solved by Dijkstra's Algorithm. Typically, when studying problems we do not need to explicitly find solutions, but merely determine if solutions exist and try and find bounds on the most efficient ways to find those solutions. In particular, we will focus on two questions:

1. Can we find a solution to a problem in polynomial time (i.e. can we find the solution in $O(n^k)$ time for some $k \in \mathbb{Z}^+$).

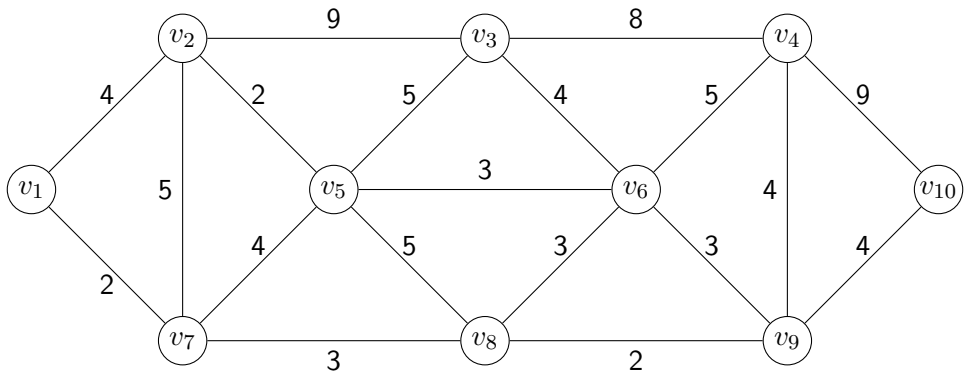
2. Can we *verify* a provided solution in polynomial time. That is, if we were given a solution, can we verify the correctness/incorrectness of that solution in polynomial time?

For every problem we have listed above the answer to both of these questions is yes. Note that our method for finding the nth Fibonacci number was not polynomial time, but an algorithm can implemented that takes $O(n)$ time by using a simple loop. The main (unanswered) question that drives the entire field NP Completness is

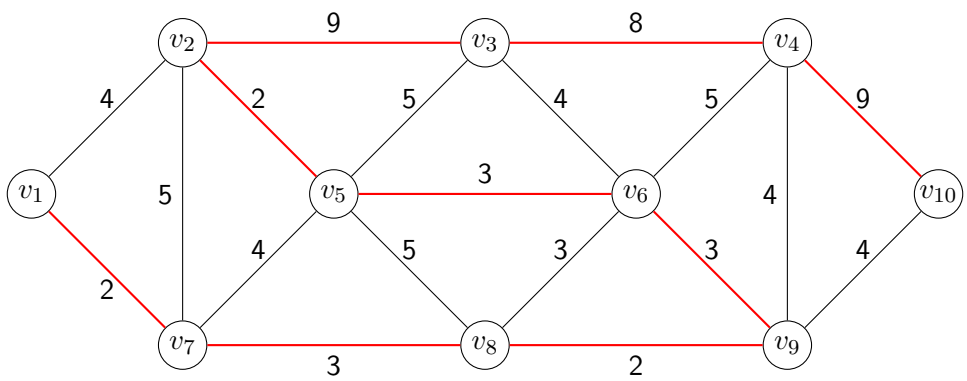
If we can **verify** a solution in polynomial time, is it possible to **find** a solution in polynomial time?

Let us begin our exploration with an example of a "hard problem" to solve in polynomial time.

Travelling Salesman Problem Given a graph G and vertices v_1 and v_n in that graph, what is the minimum weight path from v_1 to v_n that visits every vertex exactly once?



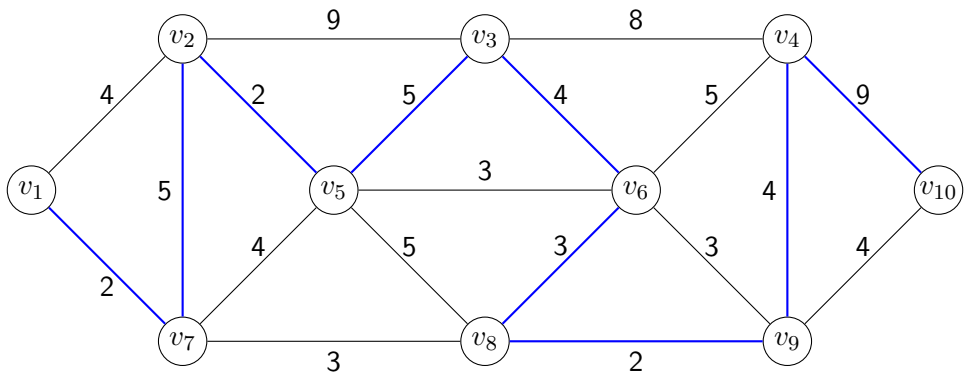
This question seems very similar to our Shortest Path question since it wants to find a shortest path but with more restrictions on the possible solutions, therefore a very natural algorithm for solving this problem might be a greedy one. Suppose that whenever we always choose to add edges to the graph that have the lowest possible edge weight and that we want to find the path from v_1 to v_{10} that visits every vertex exactly once and has the minimum weight. Doing so we arrive at the following path (we leave it to the reader to verify this),



The total weight of this path is

$$2 + 3 + 2 + 3 + 3 + 2 + 9 + 8 + 9 = 41$$

The question then becomes: does this algorithm successfully find the travelling salesman path that connects v_1 to v_n? Unfortunately no; we can demonstrate this in a number of ways, but the simplest would be to find a path with a smaller total weight. Consider the following path through the graph,



The total weight of this path is

$$2 + 5 + 2 + 5 + 4 + 3 + 2 + 4 + 9 = 36$$

which is less than the total weight of the path we discovered by our greedy algorithm. We recommend that the reader spend a little bit of time trying to think of how you might be able to solve this problem (though not too much since it is believed to be exponential in the number of vertices of the graph). We observe that some "classic" solutions to this problem take *comically* long to find answers. Let n be the number of vertices of the graph, then

1. The brute force search method which looks at all possible paths takes approximately $O(n!)$ time and even $20!$ is so unreasonably large as the render this algorithm useless.

2. The Held-Karp algorithm (which uses dynamic programming) solves the problem in $O(n^2 2^n)$ which is a serious improvement, though still exponential.

Furthermore, it is not known whether *any* algorithm can be found that will take $O(a^n)$ time where $a < 2$. Notice that these algorithms must work on *any* graph; there are certain graphs for which finding a solution is trivial. This problem appears like it might be too hard for us, so what if we ask a simpler, similar problem? The first problem was an optimization problem,

Optimization problem Given a graph G and vertices v_1 and v_n in that graph, what is the minimum weight path from v_1 to v_n that visits every vertex exactly once?

What if we simplify this problem to be a decision problem instead:

Decision problem Given a graph G, a cost C, and vertices v_1 and v_n in that graph, is there a path from v_1 to v_n which visits each vertex exactly once and has cost less than C?

This problem turns out to be equivalently difficult.....

What if instead we just remove the edge weights all together and just try to find *any* path from v_1 to v_n that visits every vertex exactly once?

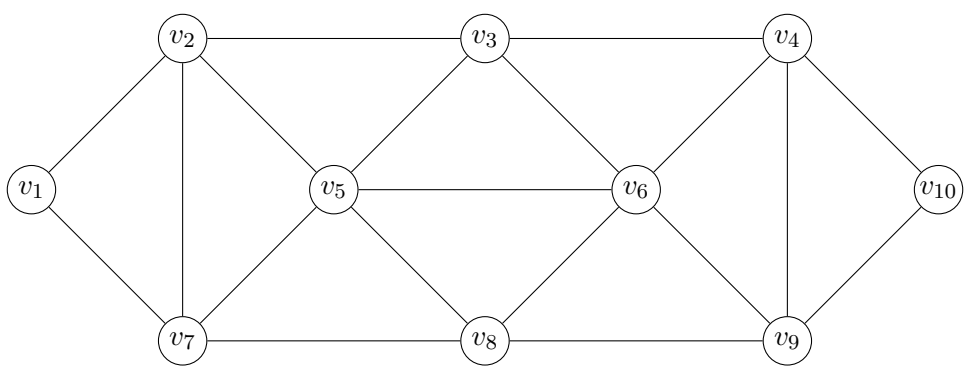
Hamiltonian Path Problem Given a graph G and vertices v_1 and v_n in that graph, is there a path from v_1 to v_n that visits every vertex exactly once?

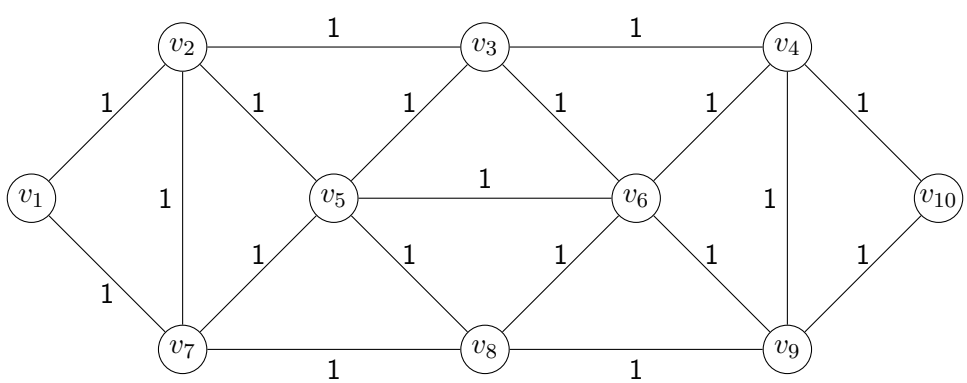
One formal mechanism for discussing how problems relate is the notion of reduction from one problem to another. A reduction allows us to say "If I can solve problem A, then I can also solve problem B". This can be helpful when you know how difficult problem A is; for example if A can be solved in polynomial time, then so must B be solvable in polynomial time.

Definition 1. A **reduction** of decision problem Q_1 to decision problem Q_2 is a mapping of every instance q_1 of problem Q_1 to an instance q_2 of problem Q_2 such that q_1 is **yes** if and only q_2 is **yes**.

Example 1. Show that the Hamiltonian Path Problem can be reduced to the Travelling Salesman Problem.

- An instance of the Hamiltonian Path Problem is a graph G with vertices v_1 and v_n.
- Construct a weighted graph G' by adding weight 1 to all edges of G.
- An instance of the Travelling Salesman Problem is a graph G' and cost n.
- There is a path from v_1 to v_n visiting all vertices in G exactly once if and only if there is a path with cost less than n from v_1 to v_n visiting all of the vertices of G' exactly once.





Therefore, the Hamiltonian Path Problem can be reduced to the Travelling Salesman Problem.

Having built up a bit of an understanding of what we mean by "problem", we must now introduce some definitions to allow us to talk more precisely about these problems.

Definition 2. A decision problem is **verifiable** in polynomial time if for every instance with the answer **yes**, there is a solution S which we can use to check in polynomial time that the answer is **yes**.

This definition and our definition of reduction gives rise to a useful property that reductions are transitive,

Proposition 1 (Reduction Transitivity)**.** If Q_1 reduces to Q_2 in polynomial time and Q_2 reduces to Q_3 in polynomial time, then Q_1 reduces to Q_3 in polynomial time.

Proof. Let f_1 be the mapping of Q_1 to Q_2 and f_2 be the mapping of Q_2 to Q_3.
Define $f_3(q_1) = f_2(f_1(q_1))$.
Function f_3 maps $q_1 \in Q_1$ to $q_3 \in Q_3$. Since f_1 and f_2 can be computed in polynomial time, f_3 can as well (polynomials are closed under composition).
If q_1 is **yes**, then $f_1(q_1)$ is **yes**. If $f_1(q_1)$ is **yes**, then $f_3(q_1) = f_2(f_1(q_1))$ is **yes**.
If q_1 is **no**, then $f_1(q_1)$ is **no**. If $f_1(q_1)$ is **no**, then $f_3(q_1) = f_2(f_1(q_1))$ is **no**.
Therefore, Q_1 reduces to Q_3 in polynomial time. □

Definition 3. A problem $Q \in$ P if it can be solved in polynomial time.

All of the actual problems we have thus far solved in this class have been in P. This is common for many problems including all of the problems listed at the start of this section.

Definition 4. A problem $Q \in$ NP if it can be verified in polynomial time.

The set NP is typically a little more difficulty for people to understand; since P stands for polynomial, many people assume that NP must stand for "non-polynomial". This is false! The set NP refers to "non-deterministic polynomial" which (in a very loose sense) means that you could develop an algorithm that could make a "clever guess" and always find the answer in polynomial time (without explicitly stating what the clever guess is). For a much more accurate and in-depth look at non-determinism students should study Automata Theory.
Note that $P \subseteq NP$ since being able to solve a problem in polynomial ensures that we can also verify it by simply solving the problem. Further observe that showing that a problem is in NP will frequently be easy since we need only discuss how to verify a solution.
The most interesting set (and the namesake of this unit) is the set of NP-Complete problems.

Definition 5. A problem $Q \in$ NP-Complete if

1. $Q \in NP$
2. Every problem $Q' \in NP$ can be **reduced** to Q in polynomial time.

Notice that the second part of this definition is a little bit insane at first glance. In order to show that a problem is NP-Complete we must show that **every single problem in** NP **can be reduced to**

it. It is not even obvious that this should be possible, but it turns out that many, many problems are NP-Complete. Notice that if we were able to solve *any* NP-Complete problem in polynomial time, then we have successfully solved every problem in NP in polynomial time by the following algorithm

```
Reduce Q_1 to NP-Complete Problem
Solve NP-Complete Problem.
```

A very natural question might be what sort of mythical problems are in this set? We will see that many completely mundane problems are NP-Complete and will list many such problems on the following page. Our final set of interest are the so-called NP-Hard problems which are problems that need not be verifiable in polynomial time, but possess the universal reduction property of the NP-Complete problems,

Definition 6. A problem $Q \in$ NP-Hard if

1. Every problem $Q' \in$ NP can be **reduced** to Q in polynomial time.

Some NP-Complete Problems

Travelling Salesman Problem Given a graph G, a cost C, and vertices v_1 and v_n in that graph, is there a path from v_1 to v_n which visits each vertex exactly once and has cost less than C?[1]

Hamiltonian Path Problem Given a graph G and vertices v_1 and v_n in that graph, is there a path from v_1 to v_n which visits each vertex exactly once?

3-Coloring Given a graph G, is there a coloring of the vertices with 3 colors so that no two adjacent vertices have the same color?

Independent Set Given a graph G and an integer K, is there a set S of K vertices in G such that no two vertices in S are adjacent?

Clique Given a graph G and an integer K, is there a set S of K vertices in G such that every pair of vertices in S are adjacent?

Vertex Cover Given a graph G and an integer K, is there a set S of K vertices in G such that every edge is incident on at least one vertex in S.

Dominating Set Given a graph G and an integer K is there a set S of K vertices in G such that every vertex of G is adjacent (or equals) a vertex in S?

Longest Simple Path Given a weighted graph G and a distance D is there a simple path from v_1 to v_n in G whose distance is greater than D?

Subgraph Isomorphism Given two graphs G and G', is G' a subgraph of G?

Boolean Satisfiability Given a Boolean expression (i.e. $(x_1 \vee x_2) \wedge (\overline{x}_2 \vee x_3 \vee \overline{x}_4) \wedge \ldots$), is there an assignment of true or false to the variables x_i so that the expression is true?

Subset Sum Given a set of positive integers $K = \{k_1, k_2, \ldots, k_n\}$ and an integer M, is there a subset of K whose sum equals M?

Set Cover Given a set U, a collection $S_1, S_2, \ldots, S_n$ of subsets of U, and integer K, is there a subcollection of U of size K whose union is U?

Set Packing Given a collection C of finite sets and an integer M, does C contain at least K mutually dijoint sets?

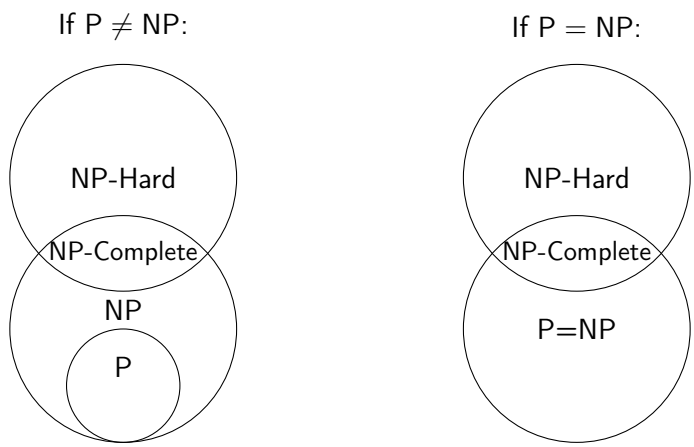
Quadratic Diophantine Equations Given integers a, b, and c, are there positive integers x and y such that $ax^2 + by = c$?

Integer Programming Given a set of linear inequalities of the form $a_{i,1}x_1 + a_{i,2}x_2 + \cdots + a_{i,n}x_n \leq b_i$, are there integers $x_1, x_2, \ldots, x_n$ which satisfy all these inequalities?

[1] This is actually NP-Hard!

Notice that many of the NP-Complete problems seem *extremely* simple at a glance. For example, solving Quadratic Diophantine Equations looks like it is simply finding the roots of a quadratic equation which we know to be extremely easy. Problems like Boolean Satisfiability look absolutely fundamental to mathematics and computer science, but are still NP-Complete.

A very natural question is how do these sets relate to each other? One of the best ways to understand how they relate is to visualize them using a Venn diagram. In order to draw an accurate visualization of this problem we must first ask the million dollar question (quite literally), does P=NP? This is a famously unsolved mathematical question which is of extreme importance to anyone who deals with computing: namely, are problems whose solutions are "easy" to check also "easy" to find? The most common belief is that P and NP are not the same, though there has been little progress proving or disproving this claim and this belief is based mostly on intuition from researchers who have worked with these problems for many years. With that in mind, let us visualize these sets supposing that P $\neq$ NP and P=NP



Notice that the picture on the right is much less interesting than the picture on the left since there is nothing all that special about the NP-Complete problems anymore. As mentioned before the more common belief is that P$\neq$NP and in that case the picture on the left has some meaning – namely, we imagine that the problems at the bottom are "easier" than the problems at the top. The NP-Complete problems are the "hardest" NP problems since solving them means that you can solve *any* NP problem. There also exist problems which are harder than any problem in NP and these are the NP-Hard problems! With these definitions under our belt we can now begin exploring various NP-Complete problems and even proving that some problems are NP-Complete! The classic starting point for this is Boolean satisfiability:

Boolean Satisfiability

Boolean Satisfiability Given a Boolean expression, is there an assignment of true and false to the variables x_i so that the expression is true?
An example of a Boolean expression is one in the form,

$$(x_1 \vee x_2) \wedge (\overline{x}_2 \vee x_3 \vee \overline{x}_4)$$

and a truth assignment is an assignment of true (T) or false (F) to each variable,

$$x_1 = T, \qquad x_2 = T, \qquad x_3 = F, \qquad x_4 = F$$

The next theorem is a famous theorem by Cook that we shall not prove because it is beyond the scope of this course,

Theorem 1 (Cook's Theorem (1971))**.** Boolean satisfiability is NP-Complete.

This theorem will allow us to show that many, many problems are NP-Complete by reducing those problems to Boolean Satisfiability (typically abbreviated SAT).
In order to more easily talk about Boolean expressions we shall introduce some terminology that will make these expressions more consistent.

Definition 7. A literal is either x_i or $\overline{x}_i$.

Definition 8. A Boolean clause (or disjunction) is an OR of literals (e.g. $x_1 \vee \overline{x}_3 \vee x_4$)

In general there are many Boolean operations that could appear in a Boolean expression (such as implication, exclusive or, and logical equivalence), however we typically only consider Boolean expressions in **Conjuctive Normal Form** (CNF); that is, the expression must be an AND of Boolean clauses, for example

$$(x_1 \vee \overline{x}_3 \vee x_4) \wedge (\overline{x}_2 \vee x_3) \wedge (\overline{x}_1 \vee x_2 \vee x_3)$$

Note that all Boolean expressions can be expressed in this form; we leave it to the reader to convince themselves of this fact by demonstrating that various Boolean operations can be re-expressed in this form.
A Boolean expression in **3-CNF** is an AND of Boolean clauses where each clause has *exactly* 3 literals.

$$(x_1 \vee \overline{x}_3 \vee x_4) \wedge (\overline{x}_2 \vee x_3 \vee \overline{x}_4) \wedge (\overline{x}_1 \vee x_2 \vee x_3)$$

A truth assignment **satisfies** a Boolean expression if it makes the Boolean expression evaluate to true.

Example 2. Give a truth assignment that satisfies

$$(\overline{x}_1 \vee x_3 \vee x_4) \wedge (x_1 \vee \overline{x}_2 \vee x_3) \wedge (x_2 \vee \overline{x}_3 \vee \overline{x}_4)$$

Proof. One way to try and develop a truth assignment for a Boolean expression is to go "clause by clause" and choose an assignment of one of the variables present in the clause to satisfy it. So, we might look at the clause

$$(\overline{x}_1 \vee x_3 \vee x_4)$$

and decide to try assigning $x_3 = T$. Then we may look at the clause

$$(x_1 \vee \overline{x}_2 \vee x_3)$$

and observe that this clause is already satisfied by $x_3 = T$. The final clause

$$(x_2 \vee \overline{x}_3 \vee \overline{x}_4)$$

can be satisfied by assigning $x_2 = T$. Therefore, any truth assignment which assigns $x_2 = x_3 = T$ will satisfy the given 3-CNF formula. □

3-SAT Problem Given a Boolean expression in 3-CNF, can it be satisfied?

Proposition 2. 3-SAT is in NP.

Proof.

$$\text{3-SAT} \subseteq \text{SAT} \in \text{NP}$$

Alternatively we can argue that □

Proposition 3. 3-SAT is in NP-Complete.

Proof. Given a CNF formula, $\varphi(x_1, x_2, \ldots, x_n)$, we must construct a 3-CNF ψ in polynomial time such that φ is satisfiable if and only if ψ is satisfiable. For example,

$$\varphi = (\overline{x}_1 \vee x_2 \vee x_3) \wedge (x_2 \vee \overline{x}_3 \vee x_4) \wedge (\overline{x}_1 \vee \overline{x}_2 \vee x_3 \vee x_4) \tag{1}$$

We introduce a new variable; let $x_A = (x_3 \vee x_4)$, and begin using our algebraic rules for manipulating

Boolean expressions to try and reduce the expression to 3-CNF. So,

$$\begin{aligned}
(\overline{x}_1 \vee \overline{x}_2 \vee x_3 \vee x_4) &= (\overline{x}_1 \vee \overline{x}_2 \vee x_A) \wedge (x_A \iff (x_3 \vee x_4)) \\
&= (\overline{x}_1 \vee \overline{x}_2 \vee x_A) \wedge (x_A \implies (x_3 \vee x_4)) \wedge (x_A \impliedby (x_3 \vee x_4)) \\
&= (\overline{x}_1 \vee \overline{x}_2 \vee x_A) \wedge (\overline{x}_A \vee (x_3 \vee x_4)) \wedge (x_A \vee \overline{(x_3 \vee x_4)}) \\
&= (\overline{x}_1 \vee \overline{x}_2 \vee x_A) \wedge (\overline{x}_A \vee x_3 \vee x_4) \wedge (x_A \vee (\overline{x}_3 \wedge \overline{x}_4)) \\
&= (\overline{x}_1 \vee \overline{x}_2 \vee x_A) \wedge (\overline{x}_A \vee x_3 \vee x_4) \wedge ((x_A \vee \overline{x}_3) \wedge (x_A \vee \overline{x}_4)) \\
&= (\overline{x}_1 \vee \overline{x}_2 \vee x_A) \wedge (\overline{x}_A \vee x_3 \vee x_4) \wedge (x_A \vee \overline{x}_3 \vee \overline{x}_3) \wedge (x_A \vee \overline{x}_4 \vee \overline{x}_4)
\end{aligned}$$

and now we can use this to write the expression in 3-CNF,

$$\begin{aligned}
\psi = (\overline{x}_1 \vee x_2 \vee x_3) \wedge (x_2 \vee \overline{x}_3 \vee x_4) \wedge (\overline{x}_1 \vee \overline{x}_2 \vee x_A) \wedge (\overline{x}_A \vee x_3 \vee x_4) \\
\wedge (x_A \vee \overline{x}_3 \vee \overline{x}_3) \wedge (x_A \vee \overline{x}_4 \vee \overline{x}_4)
\end{aligned}$$

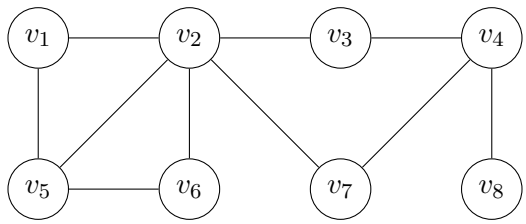
The proof for doing this in general is a bit tedious and we will not go through all of the details here. Suffice it to say, we need to generalize this process for converting any CNF expression into 3-CNF form such that the two expressions are logically equivalent by replacing any clauses with more than 3 variables with clauses that have exactly three variables. □

It is worth noting that studying SAT is a field unto itself and we have only barely begun to scratch the surface of this topic in these notes.

Independent Set

The next problem we are interested in discussing is the independent set problem,

Definition 9. An **independent set** is a subset V' of the vertices of graph $G = (V, E)$ such that if $u, v \in V'$, then $(u, v) \notin E$.



- $\{v_1\}$ is an independent set.
- $\{v_2, v_4\}$ is an independent set.
- $\{v_1, v_6, v_3, v_8\}$ is an independent set.
- $\{v_1, v_6, v_4, v_7\}$ is **not** an independent set because $(v_4, v_7) \in E$.

Independent Set Problem Given a graph G and an integer K, does G contain an independent set of size K?

We aim to show that the Independent Set Problem is NP-Complete. Therefore, we must prove two things:

1. That Independent Set is in NP.
2. That any problem in NP can be reduced to the Independent Set Problem. This is most commonly done by reducing an existing NP-Complete problem to Independent Set and by transitivity of reduction we can therefore reduce any problem in NP to Independent Set.

Proposition 4. The Independent Set Problem is in NP.

Proof. A solution to the independent set problem is a subset V' of the vertices of G. Let n be the number of vertices of G and m be the number of edges.
Given V' we first verify it contains K elements.
For each pair $u, v \in V'$, check that $(u, v) \notin E$. There are $O(n^2)$ pairs of vertices in V', so this takes $O(mn^2)$ time which is a polynomial in n and m. □

To prove that the Independent Set Problem is NP-Complete we must show that we can reduce *any* NP-Complete problem to the Independent Set Problem. We shall accomplish this by reducing 3-SAT to Independent Set.

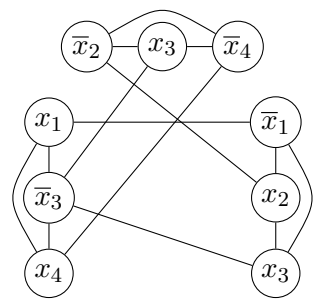
3-SAT Problem Given a Boolean expression in 3-CNF, can it be satisfied?

Proposition 5. 3-SAT reduces to the Independent Set Problem in polynomial time.

Proof. Let φ be a Boolean expression in 3-CNF. We want to express φ as a graph so that if the graph has an independent set, then φ is satisfiable,

$$\varphi = (x_1 \vee \overline{x}_3 \vee x_4) \wedge (\overline{x}_2 \vee x_3 \vee \overline{x}_4) \wedge (\overline{x}_1 \vee x_2 \vee x_3)$$

1. Create a vertex for each literal.
2. Connect each literal to the other two literals in the same clause.
3. Connect each literal x_i to $\overline{x}_i$.

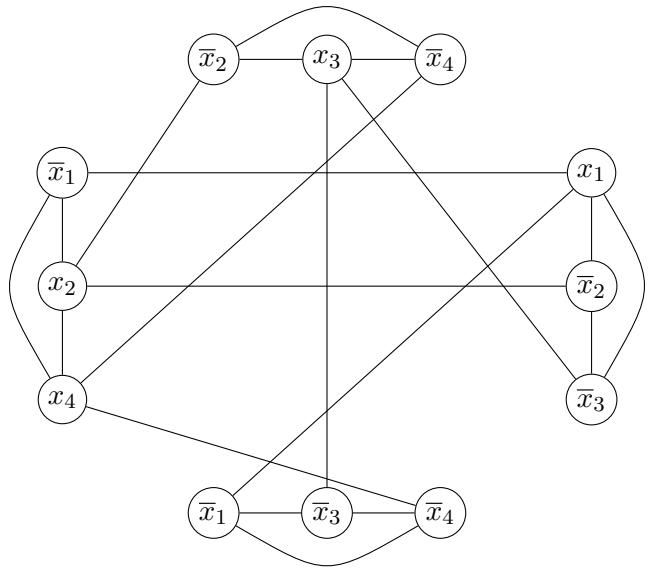


Let n be the number of variables in φ and m be the number of Boolean clauses. Note that φ can be satisfied if and only if the graph has an independent set of size m. The independent set corresponds to a truth assignment in that every vertex in the independent set is assigned to be true. Note that since every clause forms a complete subgraph in the problem that only one of the variables can be selected for the independent set. In addition to this, that the same literal can be selected multiple times for the independent set; in our graph we can see that we could select x_3 for both second and third clauses (the top cluster of vertices and the right cluster of vertices respectively). Therefore, if we manage to find an independent set of size m, each of the m clauses must be satisfied. The graph can be constructed in polynomial time so the reduction takes polynomial time. Therefore, by transitivity of reduction any problem in NP can be reduced to Independent Set in polynomial time. □

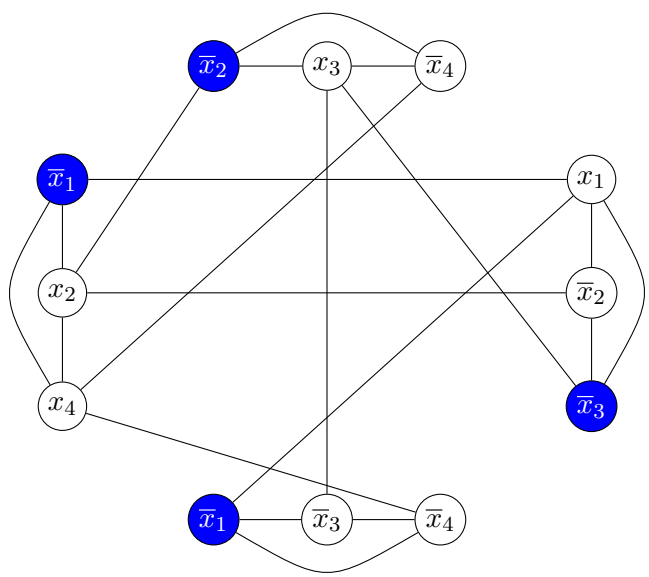
Note that this reduction does not work in reverse – not every graph can be converted into an equivalent 3-CNF statement using this algorithm. Therefore, we cannot easily see how to reduce Independent Set to 3-CNF, though such a reduction must exist since Independent Set is in NP and 3-CNF is NP-Complete.

Example 3. Express the Boolean expression as a graph and determine a truth assignment that satisfies the expression.

$$(\overline{x}_1 \vee x_2 \vee x_4) \wedge (\overline{x}_2 \vee x_3 \vee \overline{x}_4) \wedge (x_1 \vee \overline{x}_2 \vee \overline{x}_3) \wedge (\overline{x}_1 \vee \overline{x}_3 \vee \overline{x}_4) \tag{2}$$



One independent set in the graph can be demonstrated below. We have highlighted the selected nodes by coloring them blue.



This particular independent set corresponds to the truth assignment

$$x_1 = F, \qquad x_2 = F, \qquad x_3 = F, \qquad x_4 = \text{free}$$

Some students may think that they have developed some solutions to these NP-Complete problems that take polynomial time; for example

Proposition 6. An Independent Set of size 5 can be found in $O(n^5)$.

Input: Graph G with n vertices

1: **foreach** set S of 5 vertices of G **do**
2: **foreach** pair $v_i, v_j \in S$ **do**
3:   Check if (v_i, v_j) is an edge of G
4: **end for**
5: **end for**

The inner loop on lines 2-4 runs 5^2 times and we can determine if an edge is part of a graph in constant time with an adjacency matrix. If the graph has n vertices, then there are $\binom{n}{5} = \frac{n!}{(n-5)!5!}$ sets containing 5 vertices. $\binom{n}{5}$ is in $O(n^5)$, therefore, the time complexity of this algorithm is $O(n^5)$.

Proposition 7. An Independent Set of size k can be found in $O(k^2 n^k)$.

Input: Graph G with n vertices

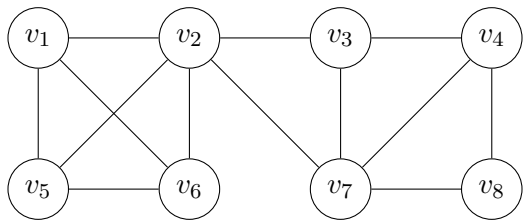
1: **foreach** set S of k vertices of G **do**
2: **foreach** pair $v_i, v_j \in S$ **do**
3:   Check if (v_i, v_j) is an edge of G
4: **end for**
5: **end for**

The inner loop on lines 2-4 runs k^2 times and we can determine if an edge is part of a graph in constant time with an adjacency matrix. If the graph has n vertices, then there are $\binom{n}{k} = \frac{n!}{(n-k)!k!}$ sets containing k vertices. $\binom{n}{k}$ is in $O(n^k)$, therefore, the time complexity of this algorithm is $O(k^2 n^k)$.
NOTE: This is **NOT** a polynomial time algorithm since k is part of the input and is not fixed.

Clique

The next NP-Complete problem that we shall discuss is another graph problem, the Clique Problem.

Definition 10. An **clique** is a subset V' of the vertices of graph $G = (V, E)$ such that if $u, v \in V'$, then $(u, v) \in E$.



- $\{v_1, v_2, v_6, v_5\}$ is a clique.
- $\{v_2, v_3, v_7\}$ is a clique.
- $\{v_3, v_4, v_7, v_8\}$ is **not** a clique since $(v_3, v_8) \notin E$.

Clique Problem Given a graph G and an integer M, does G contain a clique of size M?

As before we begin by proving that the Clique Problem can be verified in polynomial time.

Proposition 8. The clique problem is in NP.

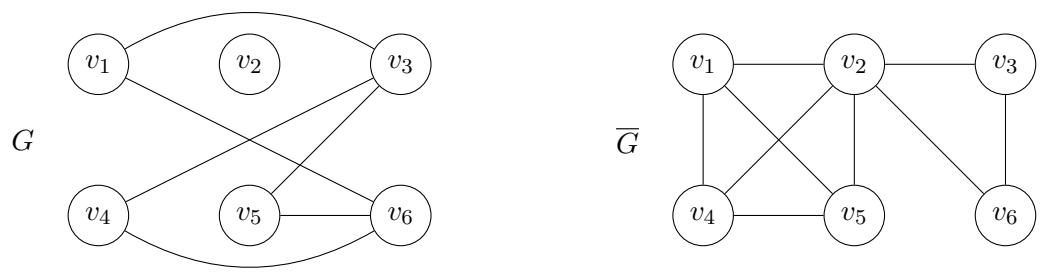
Proof. A solution to the clique problem is a subset V' of the vertices of G. Let n be the number of vertices of G and m be the number of edges.
Given V' we first verify it contains M elements.
For each pair $u, v \in V'$, check that $(u, v) \in E$. There are $O(n^2)$ pairs of vertices in V', so this takes $O(mn^2)$ time which is a polynomial in n and m. □

Next we must show that the any problem in NP can be reduced to the Clique Problem in polynomial time. We will do this by reducing a known NP-Complete Problem to Clique. Since Clique is a graph theoretic problem it stands to reason that reducing another graph theoretic problem to it should be the simplest, therefore we shall reduce the Independent Set Problem to the Clique Problem in polynomial time to demonstrate the Clique

Clique Problem Given a graph G and an integer M, does G contain a clique of size M?
Independent Set Problem Given a graph G and an integer K, does G contain an independent set of size K?
Before we can perform this reduction, we must first introduce the notion of the complement of a graph,

Proposition 9. The Independent Set Problem reduces to the Clique Problem in polynomial time.

Definition 11. The **complement** of a graph $G = (V, E)$ is a graph $\overline{G}$ with the same vertices as G such that (u, v) is an edge of $\overline{G}$ if and only if (u, v) is *not* an edge of of G.



Lemma 1. If V' is an independent set of G, then V' is a clique of $\overline{G}$

Proof. Let V' be an independent set of $G = (V, E)$. For every pair of vertices $u, v \in V'$, $(u, v) \notin E$. If $(u, v) \notin E$, then (u, v) is an edge of $\overline{G}$. That is, for every pair of vertices $u, v \in V'$, (u, v) is an edge of $\overline{G}$. So, V' is a clique of $\overline{G}$.
Suppose V' is a clique of $\overline{G}$ of size K. By definition for every pair $u, v \in V'$, we have that (u, v) is an edge of $\overline{G}$. We note that $\overline{(\overline{G})} = G$. By definition of graph complement, (u, v) is not an edge of $\overline{\overline{G}} = G$. Therefore for every pair $u, v \in V'$, (u, v) is not an edge of G so V' is an independent set of size K of G. □

With this Lemma in hand, we are not ready to perform our reduction,

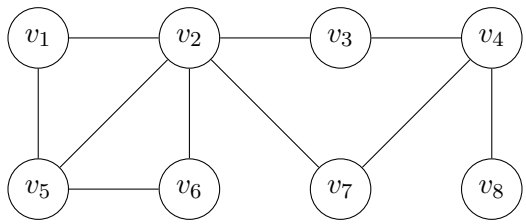
Proposition 10. The Independent Set Problem reduces to the Clique Problem in polynomial time.

Proof. Let G be a graph and K be an instance of the independent set problem. Let $\overline{G}$ be the complement of graph G.
By Lemma 1, Graph G has an independent set of size K if and only if (we didn't prove both ways, but the converse is identical) $\overline{G}$ has a clique of size K.
Mapping (G, K) to $(\overline{G}, K)$ is a reduction from the Independent Set Problem to the Clique Problem and computing $\overline{G}$ takes $O(n^2)$ time.
Therefore, Independent Set can be reduced to the Clique Problem in polynomial time; since Independent Set is NP-Complete, transitivity of reduction implies that any problem in NP can be reduced to Clique in polynomial time. □

Vertex Cover

The next NP-Complete problem that we will discuss is the Vertex Cover Problem,

Definition 12. An **vertex cover** is a subset V' of the vertices of graph $G = (V, E)$ such that for every edge $(u, v) \in E$, either $u \in V'$ or $v \in V'$.



- $\{v_1, v_3, v_5, v_6, v_7, v_8\}$ is a vertex cover.
- $\{v_2, v_4, v_5\}$ is a vertex cover.
- $\{v_2, v_3, v_8\}$ is **not** a vertex cover because edge (v_4, v_7) is not covered (i.e. v_4 and v_7 are both not in the set).

Vertex Cover Problem Given a graph G and an integer M, does G have a vertex cover of size M?

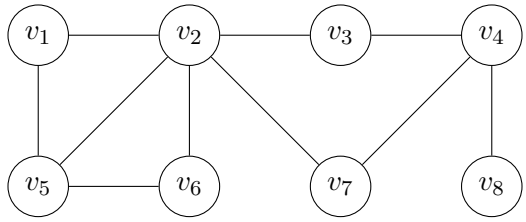
As we have several time by now we begin by proving that the Vertex Cover problem is in NP,

Proposition 11. The vertex cover problem is in NP.

Proof. A solution to the vertex cover problem is a subset V' of the vertices of G. Let n be the number of vertices of G and m be the number of edges.
Given V' we first verify it contains M elements.
For each edge $(u, v) \in E$, check that $u \in V'$ or $v \in V'$. There are $O(m)$ edges in E, so this takes $O(mn)$ time which is a polynomial in n and m. □

Next it remains to be shown that we can reduce any problem in NP to the Vertex Cover problem. We will achieve this by reducing a known NP-Complete problem to the Vertex Cover Problem. It is worth mentioning that determining *which* NP-Complete problem to reduce is part of the difficulty. Typically, we wish to find a problem that is as similar as possible to the one being studied. It is not always easy to make this determination, but playing around with definitions can sometimes be helpful. In the Clique Problem we say that Independent Set and Clique are (in a sense) inverses of each other. This lead us to explore the complement of the graph. For Vertex Cover we notice that we trying to find vertices that are "as connected as possible". A concept that seems related to that might be the Independent Set since in that problem we wanted to find vertices that were "as isolated as possible". Therefore, we shall try and reduce the Independent Set Problem to the Vertex Cover Problem!

We begin by making the following observation,



- $V' = \{v_1, v_3, v_6, v_7, v_8\}$ is an independent set.
- $V - V' = \{v_2, v_4, v_5\}$ is a vertex cover.

Lemma 2. If V' is an independent set of $G = (V, E)$, then $V - V'$ is a vertex cover of G.

Proof. Let V' be an independent set of $G = (V, E)$ and $W = V - V'$.
Let $(u, v) \in E$, since V' is an independent set either u or v are not in V'. Since $u \notin V'$ or $v \notin V'$, we have that either $u \in W$ or $v \in W$. Thus, for any edge $(u, v) \in E$, either $u \in W$ or $v \in W$, so W is a vertex cover of G □

With this Lemma, we can now see that finding an independent set is identical to finding a vertex cover with only some basic set operations differentiating the two! All that remains is to formalize this observation to finalize the proof.

Vertex Cover Problem Given a graph G and an integer M, does G have a vertex cover of size M?

Independent Set Problem Given a graph G and an integer K, does G contain an independent set of size K?

Proposition 12. The Independent Set Problem can be reduced to the Vertex Cover problem in polynomial time.

Proof. Let G be a graph and integer K be an instance of the independent set problem. Let n be the number of vertices of G and m be the number of edges.
Graph G has an independent set of size K if and only if G has a vertex cover of size $n - K$.
The mapping (G, K) to $(G, n - K)$ is a reduction of the independent set Problem to the vertex cover Problem and copying G takes $\Theta(n + m)$ time which is a polynomial in n and m. □

Set Cover

The final problem that we shall discuss is the Set Cover Problem which is a problem from Set Theory,

Definition 13. Let U be a set of elements and $C = \{S_1, S_2, \ldots, S_n\}$ be a collection of subset of U. A **set cover** is a subcollection $C' \subseteq C$ whose union is U. That is,

$$\bigcup_{x \in C'} x = U$$

$$U = \{1, 2, 3, 4, 5, 6, 7, 8\} \tag{3}$$

$$\begin{aligned} S_1 &= \{1, 2, 3\}, & S_2 &= \{1, 4, 5, 8\} \\ S_3 &= \{2, 3, 4\}, & S_4 &= \{6, 7\} \\ S_5 &= \{2, 4, 6\}, & S_6 &= \{2, 3, 5, 8\} \end{aligned}$$

- $\{S_1, S_2, S_4, S_6\}$ is a set cover.
- $\{S_2, S_3, S_4\}$ is a set cover.
- $\{S_1, S_3, S_6\}$ is **not** a set cover because $6 \notin S_1 \cup S_3 \cup S_6 = \{1, 2, 3, 4, 5, 8\}$.

Set Cover Problem Given a set U, a collection $S_1, S_2, \ldots, S_n$ of subsets of U, and integer K, is there a set cover of U of size K?

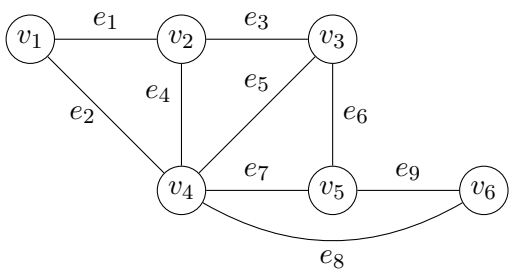
As we have 90293402 times already, we shall begin by proving that the Set Cover Problem is in NP,

Proposition 13. The Set Cover Problem is in NP.

Proof. Pretty sure we had an entire module on how to do unions good. □

Having shown that the Set Cover Problem is in NP, we must now show that any problem in NP can be reduced to the Set Cover Problem in polynomial time. We achieve this by reducing a known NP-Complete problem to the Set Cover Problem. We shall attempt to reduce the Vertex Cover Problem to the Set Cover problem since both problems seek to find a collection of objects that "covers" all of something. We hope to expand this similarity to the point of being able to prove the reduction.

Given a graph $G = (V, E)$, let $U = E$ and S_i correspond to vertex $v_i \in V$ such that if v_i is incident on e, then $e \in S_i$. For example,



$$U = \{e_1, e_2, e_3, e_4, e_5, e_6, e_7, e_8, e_9\}$$
$$S_1 = \{e_1, e_2\}$$
$$S_2 = \{e_1, e_3, e_4\}$$
$$S_3 = \{e_3, e_5, e_6\}$$
$$S_4 = \{e_2, e_4, e_5, e_7, e_8\}$$
$$S_5 = \{e_6, e_7, e_9\}$$
$$S_6 = \{e_8, e_9\}$$

Set Cover Problem Given a set U, a collection $S_1, S_2, \ldots, S_n$ of subsets of U, and integer K, is there a set cover of U of size K?

Vertex Cover Problem Given a graph G and an integer M, does G have a vertex cover of size M?

Proposition 14. The Vertex Cover Problem can be reduced to the Set Cover Problem in polynomial time.

Proof. Let graph $G = (V, E)$ and integer M be an instance of the vertex cover problem.
Let $U = E$ and let $S_i = \{e_j : v_i \text{ is incident on } e_j\}$ and C be the collection $S_1, S_2, \ldots, S_n$.
Graph G has a vertex of size M if and only if U has a set cover of size M. Note we have not proven this formally, to do so the reader must "unwind" all of the definitions for the sets and explain how this implication is true. That being said it should be clear that if V' is a vertex cover of G, then $C' = \{S_i : v_i \in V'\}$ should cover U since for every edge (v_i, v_j) either $v_i \in V'$ or $v_j \in V'$. If $v_i \in V'$, then $S_i \in C'$ and $(v_i, v_j) \in S_i$ by definition. A similar argument holds for v_j. Furthermore, this argument holds in the converse as well! Therefore, graph G has a vertex of size M if and only if U has a set cover of size M.
Mapping (G, M) to (U, C, M) is a reduction of the vertex cover problem to the set cover problem and constructing U and C takes $O(mn)$ time which is a polynomial in m and n. $\square$

All of the proofs we have performed thus far have been extremely similar in their structure. This is common in proofs on NP-Completeness; they all have a very similar flavor. In particular, they follow the following outline,

Proposition 15. Prove that problem Q_1 reduces to problem Q_2 in polynomial time.

Proof. Generically our proofs have been of the following form:

1. Let q_1 be an instance of problem Q_1.
2. Convert q_1 into an instance q_2 of problem Q_2.
3. Show that q_1 is yes if and only if q_2 is yes (we typically wrote this as a Lemma).
   (a) [q_1 yes $\implies$ q_2 yes] Suppose q_1 is yes, then ..... q_2 is yes.
   (b) [q_2 yes $\implies$ q_1 yes] Suppose q_2 is yes, then ..... q_1 is yes.
4. Explain the mapping of $q_1 \mapsto q_2$.
5. Show that the reduction takes polynomial time.

□

Nearly all of the work for proving propositions in this form comes from step 3 since it is where we must formally explain how Q_1 and Q_2 relate. This explanation and the initial conversion of q_1 into q_2 are the primary places that students seem to struggle when attempting these proofs on their own. The final step of showing that the reduction takes polynomial time is typically trivial since the process is usually just iterating over objects and copying them and/or translating information. Since this outline is a little abstract, let us study a proof that we actually performed using this outline as a skeleton.

Clique Problem Given a graph G and an integer M, does G contain a clique of size M?

Independent Set Problem Given a graph G and an integer K, does G contain an independent set of size K?

Proposition 16. Prove that the Independent Set Problem reduces to Clique Problem in polynomial time.

Proof. Following our form,

1. Let q_1 be an instance of problem Q_1.
   Let G be a graph and K be an instance of the independent set problem.

2. Convert q_1 into an instance q_2 of problem Q_2.
   Let $\overline{G}$ be the complement of graph G. $(\overline{G}, K)$ is an instance of the clique problem.

3. Show that q_1 is yes if and only if q_2 is yes.
   Graph G has an independent set of size K if and only if $\overline{G}$ has a clique of size K.

   (a) [q_1 yes $\implies$ q_2 yes] Suppose q_1 is yes, then ..... q_2 is yes.
   Suppose V' is an independent set of $G = (V, E)$ of size K. By definition for every pair $u, v \in V'$, we have that $(u, v) \notin E$. By definition of graph complement, (u, v) is an edge of $\overline{G}$. Therefore for every pair $u, v \in V'$, (u, v) is an edge of $\overline{G}$ so V' is a clique of size K of $\overline{G}$.

   (b) [q_2 yes $\implies$ q_1 yes] Suppose q_2 is yes, then ..... q_1 is yes.
   Suppose V' is a clique of $\overline{G}$ of size K. By definition for every pair $u, v \in V'$, we have that (u, v) is an edge of $\overline{G}$. We note that $\overline{(\overline{G})} = G$. By definition of graph complement, (u, v) is not an edge of $\overline{\overline{G}} = G$. Therefore for every pair $u, v \in V'$, (u, v) is not an edge of G so V' is an independent set of size K of G.

4. Explain the mapping of $q_1 \mapsto q_2$.
   Mapping (G, K) to $(\overline{G}, K)$ is a reduction from the independent set problem to the clique problem.

5. Show that the reduction takes polynomial time.
   Computing $\overline{G}$ takes $O(n^2)$ time.

$\square$